

# Pandas2

July 9, 2026

PAO25\_25\_08\_Python\_Pandas

July 08, 2026

PAO25-25 - Pandas Avanzado

Kevin Xavier Suasnavas Villarreal

[Link a mi GitHub](#)

---

## 1 1 Operaciones en Pandas

### 1.1 Búsqueda

Pandas proporciona diversas herramientas para realizar búsquedas y consultas dentro de un **DataFrame**. Estas operaciones permiten localizar columnas, verificar la existencia de determinados valores y generar máscaras booleanas que facilitan el filtrado de la información.

Entre las operaciones más utilizadas se encuentran:

- Buscar columnas mediante su nombre.
- Comprobar si determinados valores existen dentro del DataFrame.
- Contar el número de ocurrencias de un valor específico.
- Crear máscaras booleanas para aplicar filtros sobre los datos.

```
[1]: # =====  
# Importar las librerías necesarias  
# =====  
  
import pandas as pd  
import numpy as np
```

```
[2]: # =====  
# Mostrar la versión instalada de Pandas  
# =====  
  
pd.__version__
```

```
[2]: '3.0.3'
```

### 1.1.1 Búsqueda en DataFrames

Pandas ofrece diferentes herramientas para buscar información dentro de un **DataFrame**. Es posible verificar si una columna existe, localizar determinados valores mediante máscaras booleanas y contar cuántas veces aparece un dato específico.

Las funciones `isin()` y las operaciones de comparación permiten generar máscaras que facilitan el filtrado y análisis de los datos.

```
[3]: # =====  
# Crear un DataFrame con valores aleatorios  
# =====  
  
# Generar una matriz de 2 filas y 3 columnas con números aleatorios  
# entre 0 y 5.  
rand_matrix = np.random.randint(6, size=(2, 3))  
  
# Crear el DataFrame utilizando las columnas A, B y C.  
frame = pd.DataFrame(rand_matrix, columns=list("ABC"))  
  
# Mostrar el DataFrame.  
display(frame)
```

|   | A | B | C |
|---|---|---|---|
| 0 | 0 | 3 | 3 |
| 1 | 4 | 1 | 1 |

```
[4]: # =====  
# Buscar columnas dentro del DataFrame  
# =====  
  
# Verificar si existe una columna llamada "A".  
print("¿Existe la columna 'A'?", 'A' in frame)
```

¿Existe la columna 'A'? True

### 1.1.2 Búsqueda de valores

El método `isin()` permite comprobar si los valores del DataFrame pertenecen a un conjunto determinado. Como resultado se obtiene otra tabla con valores booleanos (**True** o **False**), conocida como **máscara**.

```
[5]: # =====  
# Buscar valores específicos  
# =====  
  
# Crear una máscara que indique cuáles elementos son 2 o 3.  
display(frame.isin([2, 3]))
```

|   | A     | B    | C    |
|---|-------|------|------|
| 0 | False | True | True |

```
1 False False False
```

```
[6]: # =====  
# Contar ocurrencias de un valor  
# =====  
  
# Contar cuántas veces aparece el número 4.  
print("Cantidad de valores iguales a 4:")  
print(frame.isin([4]).values.sum())  
  
# Mostrar la máscara correspondiente.  
display(frame.isin([4]))  
  
# Mostrar la máscara como un arreglo de NumPy.  
print(frame.isin([4]).values)  
  
# Mostrar el tipo del objeto retornado.  
print(type(frame.isin([4]).values))
```

Cantidad de valores iguales a 4:

1

```
      A      B      C  
0 False False False  
1  True False False  
[[False False False]  
 [ True False False]]  
<class 'numpy.ndarray'>
```

```
[7]: # =====  
# Crear una máscara utilizando operadores relacionales  
# =====  
  
# Seleccionar todos los valores mayores o iguales a 2.  
mask = frame >= 2  
  
# Contar cuántos valores cumplen la condición.  
print("Cantidad de valores mayores o iguales a 2:")  
print(mask.values.sum())  
  
# Mostrar la máscara.  
display(mask)
```

Cantidad de valores mayores o iguales a 2:

3

```
      A      B      C  
0 False  True  True  
1  True False False
```

## 1.2 Ordenación

Pandas permite ordenar fácilmente la información de un **DataFrame** mediante sus índices o los valores contenidos en las columnas. Estas operaciones facilitan el análisis y la organización de grandes conjuntos de datos.

```
[8]: # =====  
# Crear un DataFrame desordenado  
# =====  
  
from random import shuffle  
  
# Generar una matriz de 5 filas y 4 columnas.  
rand_matrix = np.random.randint(20, size=(5, 4))  
  
# Crear una lista de índices.  
indices = list(range(5))  
  
# Mezclar los índices aleatoriamente.  
shuffle(indices)  
  
# Crear el DataFrame utilizando columnas con nombres personalizados.  
frame = pd.DataFrame(rand_matrix,  
                      columns=list("DACB"),  
                      index=indices)  
  
# Mostrar el DataFrame original.  
display(frame)
```

|   | D  | A  | C  | B  |
|---|----|----|----|----|
| 3 | 18 | 16 | 4  | 14 |
| 2 | 4  | 19 | 1  | 4  |
| 0 | 0  | 18 | 13 | 19 |
| 1 | 17 | 1  | 5  | 2  |
| 4 | 10 | 11 | 8  | 11 |

```
[9]: # =====  
# Ordenar por índice  
# =====  
  
# Ordenar el DataFrame según el índice en orden descendente.  
display(frame.sort_index(ascending=False))  
  
# El DataFrame original permanece sin cambios.  
display(frame)
```

|   | D  | A  | C | B  |
|---|----|----|---|----|
| 4 | 10 | 11 | 8 | 11 |
| 3 | 18 | 16 | 4 | 14 |

|   |    |    |    |    |
|---|----|----|----|----|
| 2 | 4  | 19 | 1  | 4  |
| 1 | 17 | 1  | 5  | 2  |
| 0 | 0  | 18 | 13 | 19 |
|   | D  | A  | C  | B  |
| 3 | 18 | 16 | 4  | 14 |
| 2 | 4  | 19 | 1  | 4  |
| 0 | 0  | 18 | 13 | 19 |
| 1 | 17 | 1  | 5  | 2  |
| 4 | 10 | 11 | 8  | 11 |

```
[10]: # =====
# Ordenar las columnas
# =====

# Ordenar las columnas alfabéticamente en orden descendente.
display(frame.sort_index(axis=1, ascending=False))
```

|   |    |    |    |    |
|---|----|----|----|----|
|   | D  | C  | B  | A  |
| 3 | 18 | 4  | 14 | 16 |
| 2 | 4  | 1  | 4  | 19 |
| 0 | 0  | 13 | 19 | 18 |
| 1 | 17 | 5  | 2  | 1  |
| 4 | 10 | 8  | 11 | 11 |

```
[11]: # =====
# Ordenar las filas utilizando una columna
# =====

# Ordenar las filas según los valores de la columna A.
display(frame.sort_values(by='A', ascending=True))
```

|   |    |    |    |    |
|---|----|----|----|----|
|   | D  | A  | C  | B  |
| 1 | 17 | 1  | 5  | 2  |
| 4 | 10 | 11 | 8  | 11 |
| 3 | 18 | 16 | 4  | 14 |
| 0 | 0  | 18 | 13 | 19 |
| 2 | 4  | 19 | 1  | 4  |

```
[12]: # =====
# Ordenar las columnas utilizando los valores de una fila
# =====

# Ordenar las columnas tomando como referencia los valores
# de la fila cuyo índice es 1.
display(frame.sort_values(by=1, axis=1, ascending=True))
```

|   |    |    |   |    |
|---|----|----|---|----|
|   | A  | B  | C | D  |
| 3 | 16 | 14 | 4 | 18 |
| 2 | 19 | 4  | 1 | 4  |

|   |    |    |    |    |
|---|----|----|----|----|
| 0 | 18 | 19 | 13 | 0  |
| 1 | 1  | 2  | 5  | 17 |
| 4 | 11 | 11 | 8  | 10 |

```
[13]: # =====
# Ordenar el DataFrame y guardar los cambios
# =====

# Ordenar las filas por la columna A en orden descendente.
# inplace=True modifica directamente el DataFrame original.
frame.sort_values(by='A', ascending=False, inplace=True)

# Mostrar el resultado final.
display(frame)
```

|   |    |    |    |    |
|---|----|----|----|----|
|   | D  | A  | C  | B  |
| 2 | 4  | 19 | 1  | 4  |
| 0 | 0  | 18 | 13 | 19 |
| 3 | 18 | 16 | 4  | 14 |
| 4 | 10 | 11 | 8  | 11 |
| 1 | 17 | 1  | 5  | 2  |

### 1.3 Ranking

El **ranking** permite asignar una posición relativa a cada valor dentro de una Serie o DataFrame. Esta herramienta es útil para comparar datos, identificar los valores más altos o más bajos y establecer clasificaciones.

Pandas proporciona el método `rank()`, el cual admite diferentes estrategias para manejar valores repetidos mediante el parámetro `method`.

```
[14]: # =====
# Mostrar el DataFrame actual
# =====

# Visualizar el DataFrame sobre el que se realizará el ranking.
display(frame)
```

|   |    |    |    |    |
|---|----|----|----|----|
|   | D  | A  | C  | B  |
| 2 | 4  | 19 | 1  | 4  |
| 0 | 0  | 18 | 13 | 19 |
| 3 | 18 | 16 | 4  | 14 |
| 4 | 10 | 11 | 8  | 11 |
| 1 | 17 | 1  | 5  | 2  |

```
[15]: # =====
# Construir un ranking por filas
# =====
```

```
# Calcular el ranking de los valores de cada fila.
# method='max' asigna el rango más alto cuando existen empates.
# axis=1 indica que el cálculo se realiza horizontalmente (por filas).
display(frame.rank(method='max', axis=1))
```

|   | D   | A   | C   | B   |
|---|-----|-----|-----|-----|
| 2 | 3.0 | 4.0 | 1.0 | 3.0 |
| 0 | 1.0 | 3.0 | 2.0 | 4.0 |
| 3 | 4.0 | 3.0 | 1.0 | 2.0 |
| 4 | 2.0 | 4.0 | 1.0 | 4.0 |
| 1 | 4.0 | 1.0 | 3.0 | 2.0 |

### 1.3.1 Ordenar valores de una columna

También es posible ordenar una columna de mayor a menor y recorrer sus valores mediante un ciclo `for`. Esto resulta útil cuando se desea procesar cada dato de forma individual después de haber sido ordenado.

```
[16]: # =====
# Recorrer una columna ordenada de mayor a menor
# =====

# Ordenar el DataFrame utilizando la columna C en orden descendente.
# Posteriormente se obtiene únicamente dicha columna y se recorren
# sus valores uno por uno.

for valor in frame.sort_values(by='C', ascending=False)['C'].values:
    print(valor)
```

```
13
8
5
4
1
```

## 2 Operaciones

### 2.1 Operaciones matemáticas entre objetos

Pandas permite realizar operaciones matemáticas entre **Series** y **DataFrames** de forma sencilla. Estas operaciones pueden efectuarse utilizando los operadores tradicionales (+, -, \*, /) o mediante métodos específicos como `add()`, `sub()`, `mul()` y `div()`.

Cuando dos estructuras tienen las mismas dimensiones, las operaciones se realizan elemento por elemento. Si sus dimensiones son diferentes, Pandas alinea automáticamente filas y columnas utilizando sus índices y etiquetas.

```
[17]: # =====
# Crear dos DataFrames con valores aleatorios
```

```
# =====

# Generar dos matrices de 4x4 con números aleatorios entre 0 y 99.
matrixA = np.random.randint(100, size=(4, 4))
matrixB = np.random.randint(100, size=(4, 4))

# Convertir las matrices en DataFrames.
frameA = pd.DataFrame(matrixA)
frameB = pd.DataFrame(matrixB)

# Mostrar ambos DataFrames.
display(frameA)
display(frameB)
```

|   | 0  | 1  | 2  | 3  |
|---|----|----|----|----|
| 0 | 45 | 9  | 36 | 49 |
| 1 | 90 | 46 | 85 | 84 |
| 2 | 59 | 11 | 28 | 78 |
| 3 | 85 | 53 | 81 | 39 |

|   | 0  | 1  | 2  | 3  |
|---|----|----|----|----|
| 0 | 98 | 51 | 2  | 45 |
| 1 | 30 | 22 | 85 | 20 |
| 2 | 53 | 1  | 42 | 46 |
| 3 | 87 | 79 | 32 | 86 |

```
[18]: # =====
# Suma mediante operadores y mediante métodos
# =====

# Comparar si ambas formas de realizar la suma producen el mismo resultado.
display(frameA + frameB == frameA.add(frameB))

# Mostrar el resultado de la suma.
display(frameA + frameB)
```

|   | 0    | 1    | 2    | 3    |
|---|------|------|------|------|
| 0 | True | True | True | True |
| 1 | True | True | True | True |
| 2 | True | True | True | True |
| 3 | True | True | True | True |

|   | 0   | 1   | 2   | 3   |
|---|-----|-----|-----|-----|
| 0 | 143 | 60  | 38  | 94  |
| 1 | 120 | 68  | 170 | 104 |
| 2 | 112 | 12  | 70  | 124 |
| 3 | 172 | 132 | 113 | 125 |



```
[19]: # =====
# Resta mediante operadores y mediante métodos
# =====

# Comparar si ambas formas de realizar la resta producen el mismo resultado.
display(frameB - frameA == frameB.sub(frameA))

# Mostrar el resultado de la resta.
display(frameB - frameA)
```

```
      0      1      2      3
0  True  True  True  True
1  True  True  True  True
2  True  True  True  True
3  True  True  True  True
```

```
      0      1      2      3
0  53  42 -34  -4
1 -60 -24   0 -64
2  -6 -10  14 -32
3   2   26 -49  47
```

### 2.1.1 Operaciones entre DataFrames con diferentes dimensiones

Cuando dos DataFrames no poseen el mismo número de filas o columnas, Pandas intenta alinear automáticamente los datos utilizando los índices y nombres de las columnas. Los elementos que no tienen correspondencia se representan con el valor **NaN**.

```
[20]: # =====
# Operaciones entre DataFrames de distinto tamaño
# =====

# Crear un DataFrame de 3x3.
frameC = pd.DataFrame(np.random.randint(100, size=(3, 3)))

# Mostrar ambos DataFrames.
display(frameA)
display(frameC)

# Realizar la suma.
# Los elementos sin correspondencia aparecerán como NaN.
display(frameC + frameA)
```

```
      0      1      2      3
0  45   9  36  49
1  90  46  85  84
2  59  11  28  78
3  85  53  81  39
```

```
      0      1      2
```

|   |    |    |    |
|---|----|----|----|
| 0 | 99 | 70 | 51 |
| 1 | 18 | 58 | 30 |
| 2 | 30 | 62 | 14 |

|   |       |       |       |     |
|---|-------|-------|-------|-----|
|   | 0     | 1     | 2     | 3   |
| 0 | 144.0 | 79.0  | 87.0  | NaN |
| 1 | 108.0 | 104.0 | 115.0 | NaN |
| 2 | 89.0  | 73.0  | 42.0  | NaN |
| 3 | NaN   | NaN   | NaN   | NaN |

```
[21]: # =====
# Utilizar un valor por defecto en operaciones
# =====

# fill_value=0 reemplaza los valores inexistentes por cero
# antes de realizar la operación.
display(frameA.add(frameC, fill_value=0))
```

|   |       |       |       |      |
|---|-------|-------|-------|------|
|   | 0     | 1     | 2     | 3    |
| 0 | 144.0 | 79.0  | 87.0  | 49.0 |
| 1 | 108.0 | 104.0 | 115.0 | 84.0 |
| 2 | 89.0  | 73.0  | 42.0  | 78.0 |
| 3 | 85.0  | 53.0  | 81.0  | 39.0 |

### 2.1.2 Operadores aritméticos y tipos de datos

Las operaciones matemáticas únicamente pueden realizarse sobre datos numéricos. Si un DataFrame contiene cadenas de texto u otros tipos incompatibles, Pandas generará una excepción indicando que la operación no es válida.

```
[22]: # =====
# Ejemplo de operación no válida
# =====

# Crear un DataFrame con valores de tipo texto.
frameD = pd.DataFrame({
    0: ['a', 'b'],
    1: ['d', 'f']
})

display(frameD)

# Intentar realizar una resta entre un DataFrame numérico
# y otro de texto producirá un error.
# Se utiliza try-except para mostrar el mensaje sin detener
# la ejecución del Notebook.

try:
    display(frameA - frameD)
```

```
except Exception as e:
    print("Error:", e)
```

```

    0  1
0  a  d
1  b  f
```

Error: unsupported operand type(s) for -: 'int' and 'str'

## 2.2 Operaciones entre Series y DataFrames

Las filas y columnas de un DataFrame son objetos de tipo **Series**. Gracias a ello, es posible realizar operaciones matemáticas entre una Serie y un DataFrame completo. Pandas se encarga de alinear automáticamente los datos utilizando sus índices y nombres de columnas.

```
[23]: # =====
# Crear un DataFrame para operaciones con Series
# =====

# Generar una matriz de 3 filas y 4 columnas.
rand_matrix = np.random.randint(10, size=(3, 4))

# Crear el DataFrame con nombres de columnas.
df = pd.DataFrame(rand_matrix, columns=list("ABCD"))

# Mostrar el DataFrame.
display(df)

# Mostrar la primera fila como una Serie.
display(df.iloc[0])

# Mostrar el tipo del objeto.
display(type(df.iloc[0]))
```

```

   A  B  C  D
0  9  4  7  4
1  4  5  4  4
2  8  8  1  2
```

```

A    9
B    4
C    7
D    4
```

Name: 0, dtype: int32

pandas.Series

```
[24]: # =====
# Restar una Serie a todas las filas del DataFrame
# =====
```

```
# Restar la primera fila al resto del DataFrame.
display(df - df.iloc[0])

# La operación anterior también puede realizarse mediante
# el método sub().
display(df.sub(df.iloc[0], axis=1))
```

```
   A  B  C  D
0  0  0  0  0
1 -5  1 -3  0
2 -1  4 -6 -2
```

```
   A  B  C  D
0  0  0  0  0
1 -5  1 -3  0
2 -1  4 -6 -2
```

```
[25]: # =====
# Restar una columna a todas las filas
# =====

# Restar los valores de la columna A a todas las columnas
# del DataFrame.
display(df.sub(df["A"], axis=0))
```

```
   A  B  C  D
0  0 -5 -2 -5
1  0  1  0  0
2  0  0 -7 -6
```

**Nota:** Debido a que Pandas está construido sobre **NumPy**, la mayoría de los operadores matemáticos unarios y binarios disponibles en NumPy también pueden utilizarse sobre objetos de Pandas, facilitando el procesamiento y análisis de datos numéricos.

## 2.3 Operadores unarios y binarios

Pandas está construido sobre **NumPy**, por lo que la mayoría de los operadores matemáticos y funciones se pueden aplicar directamente sobre **Series** y **DataFrames**.

### 2.3.1 Operadores unarios

| Tipo   | Operación                                                 | Descripción                                |
|--------|-----------------------------------------------------------|--------------------------------------------|
| Unario | <code>abs</code>                                          | Valor absoluto de cada elemento.           |
| Unario | <code>sqrt</code>                                         | Raíz cuadrada de cada elemento.            |
| Unario | <code>exp</code>                                          | Calcula ( $e^x$ ) para cada elemento.      |
| Unario | <code>log</code> , <code>log10</code> , <code>log2</code> | Logaritmos en distintas bases.             |
| Unario | <code>sign</code>                                         | Retorna el signo del elemento (-1, 0 o 1). |

| Tipo   | Operación                           | Descripción                         |
|--------|-------------------------------------|-------------------------------------|
| Unario | <code>ceil</code>                   | Redondea hacia arriba.              |
| Unario | <code>floor</code>                  | Redondea hacia abajo.               |
| Unario | <code>isnan</code>                  | Indica si un elemento es NaN.       |
| Unario | <code>cos, sin, tan</code>          | Funciones trigonométricas.          |
| Unario | <code>arccos, arcsin, arctan</code> | Funciones trigonométricas inversas. |

### 2.3.2 Operadores binarios

| Tipo    | Operación                                             | Descripción                                                                                                            |
|---------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Binario | <code>add</code>                                      | Suma dos DataFrames o Series.                                                                                          |
| Binario | <code>subtract</code>                                 | Resta dos DataFrames o Series.                                                                                         |
| Binario | <code>multiply</code>                                 | Multiplica dos DataFrames o Series.                                                                                    |
| Binario | <code>divide</code>                                   | Divide dos DataFrames o Series.                                                                                        |
| Binario | <code>maximum, minimum</code>                         | Obtiene el máximo o mínimo entre pares de elementos.                                                                   |
| Binario | <code>equal, not_equal</code>                         | Compara igualdad o diferencia entre elementos.                                                                         |
| Binario | <code>greater, greater_equal, less, less_equal</code> | Comparaciones ( <code>&gt;</code> , <code>&gt;=</code> , <code>&lt;</code> , <code>&lt;=</code> ) elemento a elemento. |

### 2.3.3 Aplicación de funciones a medida con lambda

```
[26]: # =====
# Crear un DataFrame con valores enteros aleatorios.
# =====

rand_matrix = np.random.randint(10, size=(3, 4))

frame = pd.DataFrame(rand_matrix, columns=list("ABCD"))

display(frame)

# Aplicar una función lambda a cada fila (axis=1).
# La función calcula la diferencia entre el valor máximo y el mínimo.
print(frame.apply(lambda x: x.max() - x.min(), axis=1))
```

```
   A  B  C  D
0  5  6  9  2
1  3  0  3  1
2  4  3  2  7

0    7
1    3
2    5
dtype: int32
```

```
[27]: # =====
# Definir una función que calcule la diferencia entre
# el valor máximo y el mínimo de una Serie.
# =====

def max_min(x):
    return x.max() - x.min()

# Aplicar la función a cada columna (axis=0).
print(frame.apply(max_min, axis=0))
```

```
A    2
B    6
C    7
D    6
dtype: int32
```

```
[28]: # =====
# Crear un nuevo DataFrame para calcular la diferencia
# entre el valor máximo y mínimo de cada fila.
# =====

rand_matrix = np.random.randint(10, size=(3, 4))

frame = pd.DataFrame(rand_matrix, columns=list("ABCD"))

display(frame)

# Aplicar una función lambda sobre las filas (axis=1).
# Devuelve la diferencia entre el valor máximo y el mínimo de cada fila.
print(frame.apply(lambda x: x.max() - x.min(), axis=1))
```

```
   A  B  C  D
0  6  0  8  6
1  2  4  2  2
2  5  1  7  7

0    8
1    2
2    6
dtype: int32
```

### 3 3 Estadística descriptiva

- Permite realizar un análisis preliminar de los datos.
- Disponible tanto para **Series** como para **DataFrames**.

| Operación      | Descripción                                     |
|----------------|-------------------------------------------------|
| count          | Número de valores distintos de NaN.             |
| describe       | Muestra un resumen estadístico de los datos.    |
| min, max       | Valor mínimo y máximo.                          |
| argmin, argmax | Índices posicionales del valor mínimo y máximo. |
| idxmin, idxmax | Índices semánticos del valor mínimo y máximo.   |
| sum            | Suma de los elementos.                          |
| mean           | Media aritmética de los elementos.              |
| median         | Mediana de los elementos.                       |
| mad            | Desviación absoluta media.                      |
| var            | Varianza de los elementos.                      |
| std            | Desviación estándar de los elementos.           |
| cumsum         | Suma acumulada de los elementos.                |
| diff           | Diferencia entre elementos consecutivos.        |

```
[29]: # =====
# Crear un DataFrame con diferentes tipos de datos
# y obtener un resumen estadístico.
# =====

diccionario = {
    "nombre": ["Marisa", "Laura", "Manuel", "Carlos"],
    "edad": [34, 34, 11, 30],
    "puntos": [98, 12, 98, np.nan],
    "genero": ["F", "F", "M", "M"]
}

frame = pd.DataFrame(diccionario)

display(frame)

# Mostrar un resumen estadístico de las columnas numéricas.
display(frame.describe())
```

```

    nombre  edad  puntos  genero
0  Marisa    34    98.0      F
1   Laura    34    12.0      F
2  Manuel    11    98.0      M
3  Carlos    30     NaN      M
```

```

           edad  puntos
count  4.000000  3.000000
mean   27.250000 69.333333
std    10.996211 49.652123
min    11.000000 12.000000
25%    25.250000 55.000000
50%    32.000000 98.000000
```

```
75%    34.000000  98.000000
max    34.000000  98.000000
```

```
[30]: # =====
# Aplicar operaciones básicas sobre el DataFrame.
# =====

# Suma de cada columna.
print(frame.sum())

display(frame)

# Suma de los valores numéricos por fila.
print(frame.sum(axis=1, numeric_only=True))
```

```
nombre    MarisaLauraManuelCarlos
edad              109
puntos              208.0
genero              FFMM
dtype: object
```

```
   nombre  edad  puntos  genero
0  Marisa   34    98.0       F
1  Laura   34    12.0       F
2  Manuel  11    98.0       M
3  Carlos  30     NaN       M
```

```
0    132.0
1     46.0
2    109.0
3     30.0
dtype: float64
```

```
[31]: # =====
# Calcular la media de las columnas numéricas.
# =====

display(frame.mean(numeric_only=True))
```

```
edad      27.250000
puntos    69.333333
dtype: float64
```

```
[32]: # =====
# Calcular la suma acumulada de cada columna.
# =====

display(frame.cumsum())
```

```
nombre  edad  puntos  genero
```



|   |                         |     |       |      |
|---|-------------------------|-----|-------|------|
| 0 | Marisa                  | 34  | 98.0  | F    |
| 1 | MarisaLaura             | 68  | 110.0 | FF   |
| 2 | MarisaLauraManuel       | 79  | 208.0 | FFM  |
| 3 | MarisaLauraManuelCarlos | 109 | NaN   | FFMM |

```
[33]: # =====
# Contar la cantidad de valores no nulos por fila.
# =====

display(frame.count(axis=1))
```

```
0    4
1    4
2    4
3    3
dtype: int64
```

```
[34]: # =====
# Calcular la desviación estándar de la columna "edad".
# =====

print(frame["edad"].std())
```

```
10.996211468804457
```

```
[35]: # =====
# Obtener el índice del valor máximo de la columna "edad".
# =====

print(frame["edad"].idxmax())
```

```
0
```

```
[36]: # =====
# Obtener el índice del valor mínimo de la columna "puntos".
# =====

print(frame["puntos"].idxmin())
```

```
1
```

```
[37]: # =====
# Mostrar las filas que contienen el valor máximo
# de la columna "puntos".
# =====

print(frame["puntos"].max())

display(frame[frame["puntos"] == frame["puntos"].max()])
```

98.0

|   | nombre | edad | puntos | genero |
|---|--------|------|--------|--------|
| 0 | Marisa | 34   | 98.0   | F      |
| 2 | Manuel | 11   | 98.0   | M      |

```
[41]: # =====  
# Crear una nueva columna con el ranking de los puntajes.  
# =====  
  
frame["ranking"] = frame["puntos"].rank(method="max")
```

```
[40]: # =====  
# Mostrar el DataFrame actualizado con la columna ranking.  
# =====  
  
display(frame)
```

|   | nombre | edad | puntos | genero | ranking |
|---|--------|------|--------|--------|---------|
| 0 | Marisa | 34   | 98.0   | F      | 3.0     |
| 1 | Laura  | 34   | 12.0   | F      | 1.0     |
| 2 | Manuel | 11   | 98.0   | M      | 3.0     |
| 3 | Carlos | 30   | NaN    | M      | NaN     |

### 3.0.1 3.1 Agregaciones

Las **agregaciones** permiten resumir la información de un conjunto de datos agrupándolos por una o varias columnas. Entre las operaciones más comunes se encuentran el cálculo del promedio, mínimo, máximo y conteo de registros.

```
[42]: # =====  
# Agrupar los datos por el género y contar los valores  
# de cada columna dentro de cada grupo.  
# =====  
  
display(frame)  
  
df = frame.groupby("genero").count()  
  
display(df)
```

|   | nombre | edad | puntos | genero | ranking |
|---|--------|------|--------|--------|---------|
| 0 | Marisa | 34   | 98.0   | F      | 3.0     |
| 1 | Laura  | 34   | 12.0   | F      | 1.0     |
| 2 | Manuel | 11   | 98.0   | M      | 3.0     |
| 3 | Carlos | 30   | NaN    | M      | NaN     |

|        | nombre | edad | puntos | ranking |
|--------|--------|------|--------|---------|
| genero |        |      |        |         |

|   |   |   |   |   |
|---|---|---|---|---|
| F | 2 | 2 | 2 | 2 |
| M | 2 | 2 | 1 | 1 |

```
[43]: # =====
# Agrupar por la columna "puntos".
# Los valores NaN no forman parte de ningún grupo.
# =====

df = frame.groupby("puntos").count()

display(df)
```

|        | nombre | edad | genero | ranking |
|--------|--------|------|--------|---------|
| puntos |        |      |        |         |
| 12.0   | 1      | 1    | 1      | 1       |
| 98.0   | 2      | 2    | 2      | 2       |

```
[44]: # =====
# Calcular el promedio de las columnas numéricas
# agrupando por género.
# =====

display(frame.groupby("genero").mean(numeric_only=True))
```

|        | edad | puntos | ranking |
|--------|------|--------|---------|
| genero |      |        |         |
| F      | 34.0 | 55.0   | 2.0     |
| M      | 20.5 | 98.0   | 3.0     |

```
[45]: # =====
# Obtener el valor máximo de cada columna
# agrupando por género.
# =====

display(frame.groupby("genero").max())
```

|        | nombre | edad | puntos | ranking |
|--------|--------|------|--------|---------|
| genero |        |      |        |         |
| F      | Marisa | 34   | 98.0   | 3.0     |
| M      | Manuel | 30   | 98.0   | 3.0     |

```
[46]: # =====
# Aplicar varias funciones estadísticas a múltiples columnas.
# =====

display(
    frame.groupby("genero")[["edad", "puntos"]]
    .aggregate(["min", "mean", "max"])
)
```

|        | edad |      |     | puntos |      |      |
|--------|------|------|-----|--------|------|------|
| genero | min  | mean | max | min    | mean | max  |
| F      | 34   | 34.0 | 34  | 12.0   | 55.0 | 98.0 |
| M      | 11   | 20.5 | 30  | 98.0   | 98.0 | 98.0 |

```
[47]: # =====
# Definir una función para filtrar únicamente aquellos grupos
# cuya edad promedio sea mayor a 30 años.
# =====

def media(x):
    return x["edad"].mean() > 30

display(frame)

display(frame.groupby("genero").filter(media))
```

|   | nombre | edad | puntos | genero | ranking |
|---|--------|------|--------|--------|---------|
| 0 | Marisa | 34   | 98.0   | F      | 3.0     |
| 1 | Laura  | 34   | 12.0   | F      | 1.0     |
| 2 | Manuel | 11   | 98.0   | M      | 3.0     |
| 3 | Carlos | 30   | NaN    | M      | NaN     |

  

|   | nombre | edad | puntos | genero | ranking |
|---|--------|------|--------|--------|---------|
| 0 | Marisa | 34   | 98.0   | F      | 3.0     |
| 1 | Laura  | 34   | 12.0   | F      | 1.0     |

### 3.0.2 3.2 Correlaciones

Las **correlaciones** permiten medir la relación existente entre dos variables.

Su valor puede variar entre:

- **1** → Relación positiva perfecta.
- **0** → No existe relación lineal.
- **-1** → Relación negativa perfecta.

Pandas proporciona el método `corr()` para calcular el coeficiente de correlación entre columnas numéricas.

**3.2.1 Ejemplo: Fuel Efficiency** Se utilizará el conjunto de datos **Auto MPG**, disponible en el repositorio de la Universidad de California (UCI), para analizar la relación entre distintas variables de vehículos.

Fuente: <https://archive.ics.uci.edu/ml/datasets/Auto+MPG>

```
[48]: # =====
# Importar el conjunto de datos Auto MPG desde la UCI.
# =====
```

```

import pandas as pd

path = (
    "http://archive.ics.uci.edu/ml/machine-learning-databases/"
    "auto-mpg/auto-mpg.data"
)

mpg_data = pd.read_csv(
    path,
    sep=r"\s+",
    header=None,
    names=[
        "mpg",
        "cilindros",
        "desplazamiento",
        "potencia",
        "peso",
        "aceleracion",
        "año",
        "origen",
        "nombre",
    ],
    na_values="?",
    engine="c",
)

```

```

[49]: # =====
# Mostrar una muestra aleatoria de cinco registros.
# =====

display(mpg_data.sample(5))

```

|     | mpg  | cilindros | desplazamiento | potencia | peso   | aceleracion | año | \ |
|-----|------|-----------|----------------|----------|--------|-------------|-----|---|
| 275 | 17.0 | 6         | 163.0          | 125.0    | 3140.0 | 13.6        | 78  |   |
| 289 | 16.9 | 8         | 350.0          | 155.0    | 4360.0 | 14.9        | 79  |   |
| 337 | 32.4 | 4         | 107.0          | 72.0     | 2290.0 | 17.0        | 80  |   |
| 84  | 27.0 | 4         | 97.0           | 88.0     | 2100.0 | 16.5        | 72  |   |
| 64  | 15.0 | 8         | 318.0          | 150.0    | 4135.0 | 13.5        | 72  |   |

  

|     | origen | nombre                   |
|-----|--------|--------------------------|
| 275 | 2      | volvo 264gl              |
| 289 | 1      | buick estate wagon (sw)  |
| 337 | 3      | honda accord             |
| 84  | 3      | toyota corolla 1600 (sw) |
| 64  | 1      | plymouth fury iii        |

```
[50]: # =====
# Mostrar un resumen estadístico del conjunto de datos.
# =====

display(mpg_data.describe(include="all"))
```

|        | mpg        | cilindros  | desplazamiento | potencia   | peso \      |
|--------|------------|------------|----------------|------------|-------------|
| count  | 398.000000 | 398.000000 | 398.000000     | 392.000000 | 398.000000  |
| unique | NaN        | NaN        | NaN            | NaN        | NaN         |
| top    | NaN        | NaN        | NaN            | NaN        | NaN         |
| freq   | NaN        | NaN        | NaN            | NaN        | NaN         |
| mean   | 23.514573  | 5.454774   | 193.425879     | 104.469388 | 2970.424623 |
| std    | 7.815984   | 1.701004   | 104.269838     | 38.491160  | 846.841774  |
| min    | 9.000000   | 3.000000   | 68.000000      | 46.000000  | 1613.000000 |
| 25%    | 17.500000  | 4.000000   | 104.250000     | 75.000000  | 2223.750000 |
| 50%    | 23.000000  | 4.000000   | 148.500000     | 93.500000  | 2803.500000 |
| 75%    | 29.000000  | 8.000000   | 262.000000     | 126.000000 | 3608.000000 |
| max    | 46.600000  | 8.000000   | 455.000000     | 230.000000 | 5140.000000 |

  

|        | aceleracion | año        | origen     | nombre     |
|--------|-------------|------------|------------|------------|
| count  | 398.000000  | 398.000000 | 398.000000 | 398        |
| unique | NaN         | NaN        | NaN        | 305        |
| top    | NaN         | NaN        | NaN        | ford pinto |
| freq   | NaN         | NaN        | NaN        | 6          |
| mean   | 15.568090   | 76.010050  | 1.572864   | NaN        |
| std    | 2.757689    | 3.697627   | 0.802055   | NaN        |
| min    | 8.000000    | 70.000000  | 1.000000   | NaN        |
| 25%    | 13.825000   | 73.000000  | 1.000000   | NaN        |
| 50%    | 15.500000   | 76.000000  | 1.000000   | NaN        |
| 75%    | 17.175000   | 79.000000  | 2.000000   | NaN        |
| max    | 24.800000   | 82.000000  | 3.000000   | NaN        |

```
[51]: # =====
# Mostrar información general del DataFrame.
# =====

mpg_data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   mpg                    398 non-null   float64
1   cilindros              398 non-null   int64
2   desplazamiento         398 non-null   float64
3   potencia               392 non-null   float64
4   peso                   398 non-null   float64
```

```

5  aceleracion      398 non-null    float64
6  año              398 non-null    int64
7  origen           398 non-null    int64
8  nombre           398 non-null    str
dtypes: float64(5), int64(3), str(1)
memory usage: 28.1 KB

```

### 3.2.2 Correlaciones entre variables

```

[52]: # =====
# Calcular la correlación entre el rendimiento del vehículo
# (mpg) y su peso.
# =====

print(mpg_data["mpg"].corr(mpg_data["peso"]))

```

```
-0.8317409332443354
```

```

[53]: # =====
# Calcular la correlación entre el peso del vehículo
# y su aceleración.
# =====

print(mpg_data["peso"].corr(mpg_data["aceleracion"]))

```

```
-0.41745731994039337
```

### 3.0.3 3.2.3 Correlaciones entre todas las variables

Además de calcular la correlación entre dos columnas, Pandas permite obtener una **matriz de correlación**, la cual muestra el grado de relación entre todas las variables numéricas del conjunto de datos.

```

[54]: # =====
# Calcular la matriz de correlación entre todas las
# columnas numéricas del DataFrame.
# =====

display(mpg_data.corr(numeric_only=True))

```

|                | mpg       | cilindros | desplazamiento | potencia  | peso      | \ |
|----------------|-----------|-----------|----------------|-----------|-----------|---|
| mpg            | 1.000000  | -0.775396 | -0.804203      | -0.778427 | -0.831741 |   |
| cilindros      | -0.775396 | 1.000000  | 0.950721       | 0.842983  | 0.896017  |   |
| desplazamiento | -0.804203 | 0.950721  | 1.000000       | 0.897257  | 0.932824  |   |
| potencia       | -0.778427 | 0.842983  | 0.897257       | 1.000000  | 0.864538  |   |
| peso           | -0.831741 | 0.896017  | 0.932824       | 0.864538  | 1.000000  |   |
| aceleracion    | 0.420289  | -0.505419 | -0.543684      | -0.689196 | -0.417457 |   |
| año            | 0.579267  | -0.348746 | -0.370164      | -0.416361 | -0.306564 |   |
| origen         | 0.563450  | -0.562543 | -0.609409      | -0.455171 | -0.581024 |   |

|                | aceleracion | año       | origen    |
|----------------|-------------|-----------|-----------|
| mpg            | 0.420289    | 0.579267  | 0.563450  |
| cilindros      | -0.505419   | -0.348746 | -0.562543 |
| desplazamiento | -0.543684   | -0.370164 | -0.609409 |
| potencia       | -0.689196   | -0.416361 | -0.455171 |
| peso           | -0.417457   | -0.306564 | -0.581024 |
| aceleracion    | 1.000000    | 0.288137  | 0.205873  |
| año            | 0.288137    | 1.000000  | 0.180662  |
| origen         | 0.205873    | 0.180662  | 1.000000  |

```
[55]: # =====
# Eliminar las columnas "año" y "origen" antes de calcular
# la correlación, ya que no son relevantes para este análisis.
# =====

corr_data = mpg_data.drop(
    ["año", "origen"],
    axis=1
).corr(numeric_only=True)

display(corr_data)
```

|                | mpg       | cilindros | desplazamiento | potencia  | peso \    |
|----------------|-----------|-----------|----------------|-----------|-----------|
| mpg            | 1.000000  | -0.775396 | -0.804203      | -0.778427 | -0.831741 |
| cilindros      | -0.775396 | 1.000000  | 0.950721       | 0.842983  | 0.896017  |
| desplazamiento | -0.804203 | 0.950721  | 1.000000       | 0.897257  | 0.932824  |
| potencia       | -0.778427 | 0.842983  | 0.897257       | 1.000000  | 0.864538  |
| peso           | -0.831741 | 0.896017  | 0.932824       | 0.864538  | 1.000000  |
| aceleracion    | 0.420289  | -0.505419 | -0.543684      | -0.689196 | -0.417457 |

|                | aceleracion |
|----------------|-------------|
| mpg            | 0.420289    |
| cilindros      | -0.505419   |
| desplazamiento | -0.543684   |
| potencia       | -0.689196   |
| peso           | -0.417457   |
| aceleracion    | 1.000000    |

```
[57]: # =====
# Importar Matplotlib para representar la matriz
# de correlación mediante colores.
# =====

import matplotlib.pyplot as plt
```

```
[58]: # =====
# Mostrar la matriz de correlación utilizando un gradiente
# de colores para facilitar su interpretación.
```



```
# =====

corr_data.style.background_gradient(
    cmap=plt.get_cmap("RdYlGn"),
    axis=1
)
```

[58]: <pandas.io.formats.style.Styler at 0x1e0edcd2cd0>

```
[59]: # =====
# Obtener la variable con la correlación más negativa
# respecto a cada columna.
# =====

display(
    mpg_data
    .drop(["año", "origen"], axis=1)
    .corr(numeric_only=True)
    .idxmin()
)
```

```
mpg                peso
cilindros          mpg
desplazamiento     mpg
potencia           mpg
peso               mpg
aceleracion        potencia
dtype: str
```

```
[60]: # =====
# Obtener la variable con la correlación más positiva.
# El resultado incluye la correlación de cada variable consigo misma.
# =====

display(
    mpg_data
    .drop(["año", "origen"], axis=1)
    .corr(numeric_only=True)
    .idxmax()
)
```

```
mpg                mpg
cilindros          cilindros
desplazamiento     desplazamiento
potencia           potencia
peso               peso
aceleracion        aceleracion
dtype: str
```

```
[65]: # =====
# Crear una copia de la matriz de correlación y reemplazar
# la diagonal principal por ceros.
# =====

positive_corr = (
    mpg_data
    .drop(["año", "origen"], axis=1)
    .corr(numeric_only=True)
)

# Crear una copia modificable del arreglo.
corr = positive_corr.to_numpy().copy()

# Reemplazar la diagonal principal por ceros.
np.fill_diagonal(corr, 0)

# Volver a crear el DataFrame con los mismos índices y columnas.
positive_corr = pd.DataFrame(
    corr,
    index=positive_corr.index,
    columns=positive_corr.columns
)

display(positive_corr)

display(positive_corr.idxmax())
```

|                | mpg       | cilindros | desplazamiento | potencia  | peso \    |
|----------------|-----------|-----------|----------------|-----------|-----------|
| mpg            | 0.000000  | -0.775396 | -0.804203      | -0.778427 | -0.831741 |
| cilindros      | -0.775396 | 0.000000  | 0.950721       | 0.842983  | 0.896017  |
| desplazamiento | -0.804203 | 0.950721  | 0.000000       | 0.897257  | 0.932824  |
| potencia       | -0.778427 | 0.842983  | 0.897257       | 0.000000  | 0.864538  |
| peso           | -0.831741 | 0.896017  | 0.932824       | 0.864538  | 0.000000  |
| aceleracion    | 0.420289  | -0.505419 | -0.543684      | -0.689196 | -0.417457 |

|                | aceleracion |
|----------------|-------------|
| mpg            | 0.420289    |
| cilindros      | -0.505419   |
| desplazamiento | -0.543684   |
| potencia       | -0.689196   |
| peso           | -0.417457   |
| aceleracion    | 0.000000    |

|                | aceleracion    |
|----------------|----------------|
| mpg            | aceleracion    |
| cilindros      | desplazamiento |
| desplazamiento | cilindros      |
| potencia       | desplazamiento |

```
peso                desplazamiento
aceleracion         mpg
dtype: str
```

```
[66]: # =====
# Aplicar un mapa de colores para visualizar las
# correlaciones positivas y negativas.
# =====

positive_corr.style.background_gradient(
    cmap=plt.get_cmap("RdYlGn"),
    axis=1,
    vmin=-1.0,
    vmax=1.0
)
```

```
[66]: <pandas.io.formats.style.Styler at 0x1e0edf8dd10>
```

### 3.0.4 3.3 Ejercicios

Para reforzar el uso de **Pandas**, se recomienda resolver los ejercicios disponibles en el siguiente repositorio:

#### 100 Pandas Puzzles

<https://github.com/ajcr/100-pandas-puzzles/blob/master/100-pandas-puzzles.ipynb>

```
[ ]:
```